BGSMath Graduate Course: From Paper to Proof

Session 2: From paper to Lean

Michail Karatarakis

12 November 2026

Radboud University Nijmegen & Axiomatic AI

The course

	Session	Content
1	Lean and Mathlib	What formalizing means, and does not mean; first proofs; finding things in Mathlib; reading a Lean statement
2	From paper to Lean	Writing definitions and statements; the usual mistakes; drafting statements with AI and checking them
3	AI tools	What AI did this year; an agent proving a statement, live; checking what it hands back
4	Your own theorem	Statements sent in by participants, formalized live; how to do your own at home

Each session: a lecture with live demonstrations in Lean. You redo them, and the exercises, at home.

Last time, and today

> Lean checks the **proof**. You check the **statement**.

Last time: reading

- Statements written by someone else.
- Junk values, coercions, quantifier order, precedence.
- The checklist.

Today: writing

- Your own definitions and statements.
- The usual mistakes, and tests that catch them.
- Letting an AI write the draft, and checking it.

Writing definitions

A definition, from paper

Definition

$f : \mathbb{R} \to \mathbb{R}$ is *bounded* if there is M with $|f(x)| \leq M$ for all x.

In Lean

```
def Bounded (f : ℝ → ℝ) : Prop :=
  ∃ M, ∀ x, |f x| ≤ M
```

- `def` gives a name to a statement: `Bounded f` has type `Prop`.
- The body is the paper sentence, word for word: *there is M* becomes `∃ M`, *for all x* becomes `∀ x`.
- The order of the quantifiers is the order of the sentence. *M* comes first, so it may not depend on *x*.

Test your definitions

live in Lean

```
example : Bounded Real.sin :=
  ⟨1, fun x => Real.abs_sin_le_one x⟩

example : ¬ Bounded (fun x => x) := by
  intro ⟨M, hM⟩
  have h : |(|M| + 1)| ≤ M := hM (|M| + 1)
  rw [abs_of_pos (by positivity)] at h
  linarith [le_abs_self M]
```

- A definition is right only if the examples you know come out right.
- sin is bounded, $x \mapsto x$ is not. If you cannot prove both, the definition is wrong.
- One example that *has* the property, one that does not.

Write the tests before you use the definition in a theorem.

Spot the problem (1)

Definition

A natural number n is *prime* if its only divisors are 1 and n.

Spot the problem (1)

Definition

A natural number n is *prime* if its only divisors are 1 and n.

```
def IsPrime (n : ℕ) : Prop :=
  ∀ d, d | n → d = 1 ∨ d = n

-- 1 is "prime" by this definition.
example : IsPrime 1 := by
  intro d hd
  left
  exact Nat.dvd_one.mp hd

-- But not by Mathlib's.
example : ¬ Nat.Prime 1 := Nat.not_prime_one
```

- By this definition 1 is prime. On paper "$n > 1$" goes without saying; in Lean it must be written.
- A test caught it: `IsPrime 1` should have been false.

Use Mathlib's definition

```
#print Nat.Perfect
-- def Nat.Perfect : ℕ → Prop :=
-- fun n => ∑ i ∈ n.properDivisors, i = n ∧ 0 < n
```

- *n* is *perfect* if it is the sum of its divisors smaller than *n*: $6 = 1 + 2 + 3$.
- Why `0 < n`? Without it 0 is perfect: its set of divisors smaller than 0 is empty, and the empty sum is 0.
- Mathlib's definitions have been reviewed, and their edge cases thought through. If Mathlib has the notion (`Even`, `Nat.Prime`, `Continuous`, `Monotone`, ...), use it.

If you write your own, compare

```
def MyMono (f : ℝ → ℝ) : Prop :=
  ∀ x y, x ≤ y → f x ≤ f y

example (f : ℝ → ℝ) :
    MyMono f ↔ Monotone f := Iff.rfl
```

- `Iff.rfl`: the two say the same thing, by definition.
- When they do not agree by definition, prove the equivalence. The proof is a test of your definition.
- Then use Mathlib's version in your statements, so that Mathlib's lemmas apply to them.

Writing statements

From paper to statement

1. **Types.** What is each object? A number in $\mathbb{N}$, $\mathbb{Z}$ or $\mathbb{R}$, a function, a set?
2. **Hypotheses.** Everything the paper assumes, including what it leaves implicit.
3. **Quantifiers.** What may depend on what?
4. **The conclusion**, in Mathlib's words: `Set.Icc a b` for $[a, b]$, `ContinuousOn`, `Tendsto`, ...

> The same four points as the reading checklist, from the other side.

A worked example

live in Lean

Theorem

A continuous function on $[a, b]$ attains its maximum.

- `Set.Icc a b` is $[a, b]$.
- `ContinuousOn`: continuous on the interval only.
- `∃ c ∈ s, ∀ x ∈ s`: quantifiers over a set.
- Where is `a ≤ b` on paper?

In Lean

```
theorem max_on_Icc (f : ℝ → ℝ) (a b : ℝ)
    (hab : a ≤ b)
    (hf : ContinuousOn f (Set.Icc a b)) :
    ∃ c ∈ Set.Icc a b,
      ∀ x ∈ Set.Icc a b, f x ≤ f c := by
  obtain ⟨c, hc, hmax⟩ :=
    isCompact_Icc.exists_isMaxOn
    (Set.nonempty_Icc.2 hab) hf
  exact ⟨c, hc, isMaxOn_iff.mp hmax⟩
```

Spot the problem (2)

```
-- Without a ≤ b: take a = 1, b = 0.
example : ¬ ∀ (f : ℝ → ℝ) (a b : ℝ),
    ContinuousOn f (Set.Icc a b) →
    ∃ c ∈ Set.Icc a b,
      ∀ x ∈ Set.Icc a b, f x ≤ f c := by
  intro h
  obtain ⟨c, ⟨h1, h0⟩, -⟩ :=
    h (fun _ => 0) 1 0 continuousOn_const
  linarith
```

Spot the problem (2)

```
-- Without a ≤ b: take a = 1, b = 0.
example : ¬ ∀ (f : ℝ → ℝ) (a b : ℝ),
    ContinuousOn f (Set.Icc a b) →
    ∃ c ∈ Set.Icc a b,
      ∀ x ∈ Set.Icc a b, f x ≤ f c := by
  intro h
  obtain ⟨c, ⟨h1, h0⟩, -⟩ :=
    h (fun _ => 0) 1 0 continuousOn_const
  linarith
```

- If $a > b$, the interval is empty and there is no c. Without `hab` the statement is false.
- On paper the hypothesis was hidden in the notation $[a, b]$.

> Each hypothesis should be needed: remove it, and the statement should fail.

Spot the problem (3)

Claim

For every x, $x - 1 < x$.

Spot the problem (3)

Claim

For every x, $x - 1 < x$.

```
-- No type given: Lean takes x : ℕ.
example : ¬ ∀ x, x - 1 < x := by
  intro h
  have := h 0
  omega

example : ∀ x : ℝ, x - 1 < x := by
  intro x
  linarith
```

Lean infoview

```
x : ℕ
⊢ x - 1 < x
```

- With no type given, the `1` makes Lean choose $\mathbb{N}$. There the claim fails at 0.
- Always write the type. Hover over `x` to check.

Test on small cases

live in Lean

Claim

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

```
#eval (List.range 5).map fun n =>
  (∑ i ∈ Finset.range n, i, n * (n + 1) / 2)
-- [(0, 0), (0, 1), (1, 3), (3, 6), (6, 10)]
```

- `#eval` computes both sides for $n = 0, \ldots, 4$. They differ: the statement is wrong.
- Testing costs nothing, and it works before you have a proof.

Off by one

```
example : ¬ ∀ n : ℕ,
    ∑ i ∈ Finset.range n, i = n * (n + 1) / 2 := by
  intro h
  have := h 1
  simp at this

example (n : ℕ) :
    ∑ i ∈ Finset.range (n + 1), i = n * (n + 1) / 2 := by
  rw [Finset.sum_range_id, Nat.add_sub_cancel, mul_comm]
```

- `Finset.range n` is $\{0, \ldots, n-1\}$, not $\{1, \ldots, n\}$.
- The second statement is right, and `/` on $\mathbb{N}$ does no harm here: $n(n+1)$ is even. Check that whenever you divide in $\mathbb{N}$.

Limits, the Mathlib way

Definition

$u_n \to a$ if for every $\varepsilon > 0$ there is N with $|u_n - a| < \varepsilon$ for all $n \geq N$.

In Lean

```
example : Tendsto (fun n : ℕ => 1 / (n : ℝ))
    atTop (𝓝 0) :=
  tendsto_one_div_atTop_nhds_zero_nat
```

Lean says

```
Metric.tendsto_atTop : Tendsto u atTop (𝓝 a) ↔
  ∀ ε > 0, ∃ N, ∀ n ≥ N, dist (u n) a < ε
```

- Mathlib states limits as `Tendsto u atTop (𝓝 a)`. Write your statements this way, so that Mathlib's lemmas apply.
- Lemmas like `Metric.tendsto_atTop` translate it back into the ε–N form.

Checking a statement

Can the hypotheses hold?

```
-- The hypotheses of max_on_Icc can hold together.
example : ∃ (f : ℝ → ℝ) (a b : ℝ),
    a ≤ b ∧ ContinuousOn f (Set.Icc a b) :=
  ⟨fun x => x, 0, 1, zero_le_one, continuousOn_id⟩

-- Those of everything_is_42 (session 1) cannot.
example : ¬ ∃ x : ℝ, 0 < x ∧ x < 0 := by
  rintro ⟨x, h₁, h₂⟩
  linarith
```

- Give one example where all the hypotheses hold. If there is none, the theorem says nothing (session 1, `everything_is_42`).
- This is a test of the statement, so do it before the proof.

Use it on a case you know

```
example : ∃ c ∈ Set.Icc (0 : ℝ) 1,
    ∀ x ∈ Set.Icc (0 : ℝ) 1, x ^ 2 ≤ c ^ 2 :=
  max_on_Icc (fun x => x ^ 2) 0 1 zero_le_one
    (by fun_prop)
```

- Apply the statement to an example where you know the answer.
- If this is hard, the statement may be stated in a form nobody can use.

At home `Exercises2.lean`, parts 1 and 2: write the definitions and statements, then run these tests on them.

Checking a statement: the tests

1. **Read it back.** Translate the Lean into English *without* looking at the paper, then compare.
2. **Small cases.** `#eval`, `decide`, `norm_num` on examples.
3. **The hypotheses can hold.** Give an example.
4. **Each hypothesis is needed.** Remove it, and the statement fails.
5. **A case you know.** Apply the statement to it.
6. **The negation.** If you can prove it, the statement is false.
7. **Mathlib.** If Mathlib has the result, compare with its statement.

Drafting statements with AI

What an AI can do here

Useful

- Finding Mathlib's names: `Set.Icc`, `ContinuousOn`, `Tendsto`.
- Getting a draft to compile.
- Translating a Lean statement back into English.

Check for

- The usual mistakes: a missing hypothesis, the wrong type, off by one.
- A statement changed until it is easy to prove.
- A definition of its own where Mathlib has one.

> The AI writes the draft. You are responsible for what it says.

A prompt that helps

Prompt

Formalize the following statement in Lean 4 with Mathlib. Use Mathlib's definitions; do not define your own. Do not prove it: end with `sorry`. Then:

1. translate your Lean statement back into English;
2. list each hypothesis and say why it is needed;
3. give one example where all the hypotheses hold.

Statement: ...

- Asking for `sorry` keeps the statement and the proof apart.
- Steps 1–3 are three of the tests. Check the answers yourself.

Spot the problem (4)

```
-- Paper: a monotone sequence that is bounded above converges.
theorem draft (a : ℕ → ℝ) (hmono : Monotone a)
    (hbdd : BddAbove (Set.range a))
    (hconv : ∃ L, Tendsto a atTop (𝓝 L)) :
    ∃ L, Tendsto a atTop (𝓝 L) :=
  hconv
```

Spot the problem (4)

```
-- Paper: a monotone sequence that is bounded above converges.
theorem draft (a : ℕ → ℝ) (hmono : Monotone a)
    (hbdd : BddAbove (Set.range a))
    (hconv : ∃ L, Tendsto a atTop (𝓝 L)) :
    ∃ L, Tendsto a atTop (𝓝 L) :=
  hconv
```

Lean says

```
warning: Variable name `hmono` is not explicitly referenced.
warning: Variable name `hbdd` is not explicitly referenced.
```

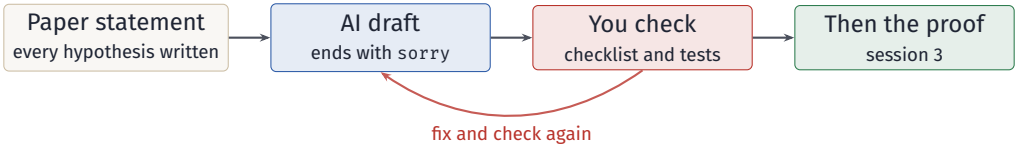
Lean accepts this The conclusion is one of the hypotheses: `hconv`. The theorem is true and says nothing.

The fix

```
theorem mono_conv (a : ℕ → ℝ) (hmono : Monotone a)
    (hbdd : BddAbove (Set.range a)) :
    ∃ L, Tendsto a atTop (𝓝 L) :=
  ⟨_, tendsto_atTop_ciSup hmono hbdd⟩
```

- Without `hconv`, the proof needs a real lemma: `tendsto_atTop_ciSup`, and the limit is $\sup_n a_n$.
- A hypothesis the proof never uses is a warning sign. Either it is not needed, or the statement is wrong.

The workflow



> Do not ask for a proof until you trust the statement.

At home

At home

`Exercises2.lean`, in the browser or on your laptop.

1. Definitions: write the `def`; its tests are below it.
2. Statements: from the paper sentence. The proof is in Mathlib.
3. Find the bug: true, false, or true for a silly reason? Fix the false ones.
4. Your own statement, from session 1. Draft it with an AI, then check it.

- First redo the demos. Their code is in `Session2.lean`.
- Part 4: note every change you make to the AI's draft, and why.
- Send me your statement: in session 4 I formalize some of them live.

> Today the stating numbers are higher than the proving numbers. That is the point.

Next time: AI tools

- AI agents that write Lean proofs, in practice.
- Case studies: how the large formalizations of the last year were done.
- An agent proving a statement, live. At home, you try it on yours.

> Before next time: Lean and Mathlib working on your laptop (see the setup sheet), and your statement from part 4 compiling with `sorry`.

Resources

- J. Avigad and P. Massot, *Mathematics in Lean*. Its chapters on sets, functions and structures show how Mathlib states things.
- K. Buzzard, *Formalising Mathematics* (Imperial College). Notes on reading and writing statements.
- The Mathlib documentation, Loogle and LeanSearch: find Mathlib's version before you write your own.
- The Lean Zulip chat, stream *#new members*.